FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

3

CASE-BASED REASONING (CBR) SYSTEM DEVELOPMENT

Student Name	Gavryl Garlitos
Section:	AN43
Professor:	Crisola G. Tan
Date:	October 24 2026

I. BACKGROUND INFORMATION

Case-Based Reasoning (CBR) is an intelligent problem-solving approach used in **Knowledge Representation and Reasoning (KRR)**. Instead of relying solely on predefined rules, CBR systems solve new problems by referring to past experiences, also known as cases. These cases contain previous problems and their successful solutions. When a new situation arises, the system identifies the most similar past case, adapts the old solution to fit the new context, and applies it. Afterward, the new case is saved, allowing the system to learn and continuously improve.

CBR is widely used in various fields, such as disaster response, legal assistance, and intelligent tutoring, because it mimics how humans naturally make decisions. For example, emergency responders rely on past typhoon data to plan evacuations; lawyers examine previous legal cases to build arguments; and tutoring systems look at prior student mistakes to provide customized guidance. In all these examples, past knowledge becomes the foundation for solving future problems more effectively.

This activity allows students to experience how CBR works by developing a simple, functional CBR program based on real-world scenarios. Through pair programming, students will learn how to represent cases, implement similarity assessment, adapt solutions, and store new learning experiences—reflecting the complete CBR cycle. This hands-on approach strengthens understanding of how intelligent systems use experiential knowledge to support reasoning, decision-making, and learning.

II. TECHNICAL ASSESSMENT

CASE-BASED REASONING (CBR) SYSTEM DEVELOPMENT

Objective

At the end of this laboratory activity, students are expected to:

1. **Understand and apply the Case-Based Reasoning (CBR) cycle**, including retrieval, reuse, revision, and retention.
2. **Represent cases effectively** using appropriate data structures such as dictionaries, arrays, JSON objects, or classes.
3. **Implement a similarity assessment method** to identify relevant past cases based on defined attributes.
4. **Adapt retrieved solutions** to meet the needs of a new problem scenario based on the chosen case study.
5. **Demonstrate learning in CBR** by saving the newly solved case into the system's case base.
6. **Develop collaboration skills** through pair programming and joint problem analysis.
7. **Apply knowledge representation and reasoning concepts** to real-world problems in disaster response, legal reasoning, or intelligent tutoring.

Materials Needed

1. Hardware
    - A laptop or desktop computer
    - Stable internet connection (for research, documentation, submission)
2. Software

A programming environment:

- Python (recommended: IDLE, VS Code, Jupyter Notebook, PyCharm)
- Any text editor (Notepad++, Sublime, VS Code) if needed

3. File Formats & Tools

Word processor for documentation:

- MS Word, Google Docs, or LibreOffice
- Screenshot tool (e.g., Snipping Tool, Lightshot, Print Screen)

4. Reference Materials

Lecture notes on:

- Knowledge Representation & Reasoning
- Case-Based Reasoning cycle
- Similarity measures

Provided case studies (Disaster DSS, Legal DSS, Intelligent Tutoring System)

5. Pair Collaboration Tools (optional but useful)
    - Google Drive for shared documentation
    - GitHub/GitLab for code collaboration

General Instructions

1. This **Technical Assessment (TA3)** must be completed by Pair.
2. Please ensure that you are grouped as pairs in the People tab.
3. Select only one case study from the list below.
4. Include clear comments in your code to describe its logic.
5. Each program should display the correct input, process, and output.
6. Ensure that your script displays clear and readable outputs.
7. Compile all Python codes and corresponding outputs into one PDF file.
8. Save it as **TA3_lastname1_lastname2.pdf**
   (Example: TA2_Patilano_Loverez.pdf)
9. Submit the **python file** and **PDF** file.

📌 Step-by-Step Instructions

1. Form a pair

Choose a partner. Each pair will work collaboratively on research, coding, debugging, and presenting your results.

2. Choose ONE (1) case study

Select only one case study from the list below. Your program must be based on the scenario and logic of the chosen system:

Case Study Options:

1. *Disaster Response Decision Support System (DSS)*
2. *AI-Based Legal Decision Support System*
3. *Intelligent Tutoring System (ITS)*

3. Define your case base

Create at least **3–5 past cases** stored as dictionaries, JSON objects, arrays, or any structure.

Example structure:

```
{
  "problem": {...},
  "solution": {...}
}
```

4. Implement similarity assessment

Your program must include a way to measure similarity between the new case and past cases.
Examples:

- Attribute matching
- Weighted scoring
- Simple rule-based comparison

Output should show: (sample)

✔ Similarity score or ranking
✔ The most relevant past case retrieved

5. Adapt the solution

Modify the retrieved solution so it fits the new problem scenario.
This part should reflect the logic of your chosen case study:

Examples:

- **DSS:** Adjust evacuation plans based on population size
- **Legal DSS:** Modify arguments due to new digital evidence
- **ITS:** Give personalized feedback based on student errors

6. Demonstrate learning (retain stage)

After generating a final solution, store the new case into your case base.

Your program must show:

✔ "New case saved" or similar confirmation
✔ Updated case list

7. Display clear program outputs

Your output must include:

- New problem description
- Retrieved similar case
- Adapted final solution
- Updated case base

8. Submission Requirements

Submit the following:

A. Program File

- Python

B. Short Documentation (1–2 pages)

Include:

- Title of chosen case study
- Description of your case base
- Explanation of similarity method
- Adaptation logic
- Screenshots of program execution

C. Pair Reflection (5–7 sentences)

Discuss:

- What each partner contributed
- What challenges you encountered
- What you learned about CBR

Case Studies:

Case Study 1: Disaster Response Decision Support System

A CBR-based DSS helps local government units respond to natural disasters.

- Past cases include typhoons, floods, and earthquakes.
- New event: *Typhoon with expected flooding in urban areas.*
- System retrieves similar events: *Typhoon Ursula (2019)* and *Typhoon Odette (2021)*.
- It adapts recommended responses such as evacuation strategy, resource allocation, and shelter capacity.
- After execution, the system stores new outcomes for improved future reasoning.

Learning Outcome:

CBR helps governments respond faster by using *experiential knowledge* rather than rebuilding decisions from scratch.

Case Study 2: AI-Based Legal Decision Support System

A legal CBR tool assists lawyers by retrieving previous court cases similar to the current legal issue.

Scenario:

Client is charged with “theft involving electronic transfer.” System retrieves past cases involving:

- online theft
- digital fraud
- unauthorized bank transfers

Adaptation:
The reasoning engine adjusts argument patterns to match the new technical context.
Learning:
After final verdict, the case is stored—enhancing the system's knowledge for future cybercrime-related offenses.

Case Study 3: Intelligent Tutoring System

A CBR system identifies the learning struggles of students.
Student submits a math solution with repeated errors.
System retrieves past students with similar error patterns (e.g., misusing formulas).
It adapts feedback and personalized lessons based on successful interventions.
Learning:
System improves by retaining new student profiles and outcomes.

Your Code - Laboratory Tasks ⇩

Case Study 3: Intelligent Tutoring System

```
import json
from pathlib import Path

# Google Colab-compatible case database
CASE_FILE = Path("/content/cases.json")

# 1. CASE REPRESENTATION
# Five sample cases containing previous student problems and solutions.
INITIAL_CASES = [
    {
        "id": 1,
        "grade": 7,
        "topic": "fractions",
        "errors": ["wrong_denominator", "addition_rule"],
        "intervention": "Review fraction addition and practise with visual
fraction bars.",
        "outcome": "Improved after guided examples."
    },
    {
        "id": 2,
        "grade": 8,
```

```
        "topic": "linear_equations",
        "errors": ["sign_error", "inverse_operation"],
          "intervention": "Practise inverse operations one step at a time
and check signs.",
        "outcome": "Solved similar equations correctly."
    },
    {
        "id": 3,
        "grade": 7,
        "topic": "fractions",
        "errors": ["simplification", "wrong_denominator"],
         "intervention": "Use equivalent fractions and practise simplifying
answers.",
        "outcome": "Made fewer fraction errors."
    },
    {
        "id": 4,
        "grade": 9,
        "topic": "algebra",
        "errors": ["sign_error", "like_terms"],
          "intervention": "Review positive and negative terms and combine
like terms.",
        "outcome": "Improved accuracy in algebra exercises."
    },
    {
        "id": 5,
        "grade": 8,
        "topic": "fractions",
        "errors": ["multiplication_rule", "simplification"],
         "intervention": "Practise fraction multiplication and simplifying
answers.",
        "outcome": "Correctly multiplied and simplified fractions."
    }
]

# Error patterns that users can select.
ERROR_OPTIONS = {
```

```
    "1": ("wrong_denominator", "Using the wrong denominator"),
    "2": ("addition_rule", "Incorrect fraction addition rule"),
    "3": ("simplification", "Difficulty simplifying fractions"),
    "4": ("sign_error", "Sign errors"),
    "5": ("inverse_operation", "Incorrect inverse operations"),
    "6": ("like_terms", "Combining unlike terms"),
    "7": ("multiplication_rule", "Incorrect fraction multiplication rule")
}


def save_cases(cases):
    """Save the case base to a JSON file."""
    with CASE_FILE.open("w", encoding="utf-8") as file:
        json.dump(cases, file, indent=4)


def load_cases():
    """Load existing cases or initialize the five sample cases."""
    if not CASE_FILE.exists():
        save_cases(INITIAL_CASES)

    try:
        with CASE_FILE.open("r", encoding="utf-8") as file:
            cases = json.load(file)

        return cases if isinstance(cases, list) else INITIAL_CASES.copy()

    except (json.JSONDecodeError, OSError):
        print("Could not read cases.json. Loading starter cases.")
        return INITIAL_CASES.copy()


# 2. SIMILARITY ASSESSMENT
def similarity(new_case, old_case):
    """
    Calculate similarity from 0 to 100.
```

```
    Topic match: 30 points
    Same grade: 20 points
    One grade apart: 10 points
    Error-pattern similarity: up to 50 points
    """
    score = 0.0

    # Compare the math topics.
    if new_case["topic"] == old_case["topic"]:
        score += 30

    # Compare the grade levels.
    grade_difference = abs(
        new_case["grade"] - old_case["grade"]
    )

    if grade_difference == 0:
        score += 20
    elif grade_difference == 1:
        score += 10

    # Compare error patterns using Jaccard similarity.
    new_errors = set(new_case["errors"])
    old_errors = set(old_case["errors"])

    union = new_errors | old_errors
    intersection = new_errors & old_errors

    if union:
        error_similarity = len(intersection) / len(union)
    else:
        error_similarity = 1

    score += 50 * error_similarity

    return round(score, 1)
```

```
# 3. CASE ADAPTATION
def adapt_solution(new_case, retrieved_case):
    """Adapt a previous intervention to the new student's errors."""

    advice = [retrieved_case["intervention"]]

    specific_advice = {
        "wrong_denominator":
            "Find a common denominator before calculating.",

        "addition_rule":
            "Add numerators only after the denominators match.",

        "simplification":
            "Simplify the answer by dividing the numerator and denominator
by a common factor.",

        "sign_error":
            "Double-check positive and negative signs at every step.",

        "inverse_operation":
            "Use the opposite operation to isolate the unknown.",

        "like_terms":
            "Combine only terms with the same variable and exponent.",

        "multiplication_rule":
             "Multiply numerators together and denominators together, then
simplify."
    }

    # Add advice that matches the student's selected errors.
    for error in new_case["errors"]:
        if error in specific_advice:
            advice.append(specific_advice[error])
```

```
    topic = new_case["topic"].replace("_", " ")

    advice.append(
        f"Suggested plan for Grade {new_case['grade']} {topic}: "
        "review one worked example, solve three guided questions, "
        "then try two questions independently."
    )

    return " ".join(advice)


def ask_student_details():
    """Collect information about the new student case."""

    print("\n=== NEW STUDENT CASE ===")

    # Get a valid grade level.
    while True:
        try:
            grade = int(input("Student grade level (7-12): "))

            if 7 <= grade <= 12:
                break

            print("Please enter a grade from 7 to 12.")

        except ValueError:
            print("Please enter a whole number.")

    # Get the math topic.
    topic = input(
        "Math topic (e.g., fractions, linear_equations, algebra): "
    ).strip().lower()

    while not topic:
                topic = input("Topic cannot be blank. Enter a topic:
").strip().lower()
```

```
    # Display available error patterns.
    print("\nSelect the student's error pattern(s):")

    for key, (_, description) in ERROR_OPTIONS.items():
        print(f"{key}. {description}")

    # Allow multiple error patterns.
    while True:
        choices = [
            item.strip()
            for item in input(
                "Enter choices separated by commas (e.g., 1,3): "
            ).split(",")
        ]

        if choices and all(choice in ERROR_OPTIONS for choice in choices):
            choices = list(dict.fromkeys(choices))
            break

        print("Invalid input. Enter numbers from 1 to 7.")

    return {
        "grade": grade,
        "topic": topic,
        "errors": [
            ERROR_OPTIONS[key][0] for key in choices
        ],
        "error_choices": choices
    }


def main():
    # Load the case base.
    cases = load_cases()

    # Describe the new problem.
```

```
    new_case = ask_student_details()

    print("\n=== 1. CASE REPRESENTATION ===")
    print("Grade:", new_case["grade"])
    print("Topic:", new_case["topic"].replace("_", " "))

    print(
        "Errors:",
        ", ".join(
            ERROR_OPTIONS[key][1]
            for key in new_case["error_choices"]
        )
    )

    # 2. Calculate scores and retrieve the closest case.
    print("\n=== 2. SIMILARITY ASSESSMENT AND RETRIEVAL ===")

    ranked = sorted(
        [
            (similarity(new_case, case), case)
            for case in cases
        ],
        key=lambda item: item[0],
        reverse=True
    )

    # Display similarity rankings.
    for rank, (score, case) in enumerate(ranked, start=1):
        print(
            f"{rank}. Case {case['id']} | "
            f"Topic: {case['topic']} | "
            f"Similarity: {score:.1f}%"
        )

    best_score, best_case = ranked[0]

    print(
```

```
        f"\nRetrieved case: Case {best_case['id']} "
        f"({best_score:.1f}% similarity)"
    )
    print("Past intervention:", best_case["intervention"])
    print("Past outcome:", best_case["outcome"])

    # 3. Reuse and adapt the retrieved solution.
    print("\n=== 3. ADAPTED SOLUTION ===")

    adapted = adapt_solution(new_case, best_case)
    print(adapted)

    # 4. Retain the new case for future use.
    print("\n=== 4. LEARNING / RETENTION ===")

    next_id = max(
        (case.get("id", 0) for case in cases),
        default=0
    ) + 1

    retained_case = {
        "id": next_id,
        "grade": new_case["grade"],
        "topic": new_case["topic"],
        "errors": new_case["errors"],
        "intervention": adapted,
        "outcome": (
            "New case retained. Update this outcome "
            "after observing student progress."
        )
    }

    cases.append(retained_case)
    save_cases(cases)

    print(
        f"New case saved successfully as Case {next_id} "
```

```
        f"in {CASE_FILE}."
    )

    print(f"\nUpdated case base contains {len(cases)} cases:")

    for case in cases:
        print(
            f"Case {case['id']}: Grade {case['grade']} | "
            f"{case['topic']} | "
            f"Errors: {', '.join(case['errors'])}"
        )


# Start the program.
main()
```

